

Solidify: Real-Time Static Analysis for Solidity in IDE

Nguyen Huu Canh, Sit Khai Dong

November 30, 2025

ABSTRACT

Smart contracts underpin decentralized finance (DeFi) and Web3 applications, stewarding billions of dollars in value. Security defects in Solidity code can cause catastrophic losses. Existing static analyzers are typically wired into continuous integration (CI/CD) pipelines and surface findings only after code is written, leaving a blind spot during development when issues could be caught immediately.

This paper introduces **Solidify**, a Visual Studio Code extension that delivers real-time static analysis directly inside the integrated development environment (IDE). The tool adopts a hybrid strategy: it leverages `tree-sitter` to construct rich Abstract Syntax Trees (ASTs) while complementing them with high-performance regular expressions for rapid pattern checks. A modular rule architecture makes it straightforward to extend coverage across security, syntax, naming conventions, and pragma versioning.

Our evaluation on 20 real-world smart contracts shows that Solidify achieves an average analysis latency of 82 ms (± 15 ms), roughly $30\times$ faster than baseline tools such as Slither (~ 2.5 s). The analyzer attains $F1 = 0.60$ (Precision: 0.61, Recall: 0.59), demonstrating that sacrificing some depth of analysis for instant feedback is both practical and impactful for developer experience.

Index Terms: Smart Contracts, Solidity, Static Analysis, Blockchain Security, Visual Studio Code, Abstract Syntax Tree (AST).

INTRODUCTION

The rise of blockchain technology ushered in a new era of decentralized applications with smart contracts at the core. Solidity has become the de facto language for authoring these contracts on Ethereum and other EVM-compatible chains. Yet the immutability of blockchain - the very property that ensures trust-cuts both ways: once deployed, smart contract code cannot be patched. Seemingly minor vulnerabilities can be exploited to drain digital assets worth millions of dollars [1].

High-profile incidents such as The DAO (2016) or the Parity Wallet hack (2017) underscore the need for rigorous code inspection. The community responded with heavyweight static analyzers like Slither [2] and Mythril [3]. These tools excel at deep analysis but are optimized for whole-project scans, often within CI/CD pipelines. Developers therefore receive feedback only after finishing significant work and pushing their changes. This slow feedback loop raises the cost of fixes and hampers productivity.

We identify a critical gap: the lack of tools that raise security and syntax warnings while a developer is typing. Immediate feedback not only accelerates remediation but also reinforces best practices by highlighting anti-patterns in context.

To close this gap we built **Solidify**, a VS Code extension designed to be lightweight, responsive, and tightly integrated with the editor, effectively acting as a real-time assistant for Solidity developers.

Our main contributions are:

- **Hybrid analysis architecture:** We present a design that combines AST-based reasoning with regex-based pattern detection.
- **Modular rule system:** We architect the analyzer so that rules can be developed, maintained, and extended independently.
- **Real-time integration:** We implement an IDE-native static analyzer that produces instant feedback without disrupting the developer’s flow.
- **Comprehensive rule set:** We deliver an initial suite that addresses high-impact security flaws, common syntax mistakes, and code quality conventions.

RELATED WORK

Security analysis of smart contracts has attracted significant research. Existing tools broadly fall into static or dynamic analysis categories.

Static analysis inspects source code without execution. **Slither** [2] is among the most capable frameworks, operating on its SlithIR intermediate representation to detect a wide range of vulnerabilities. **Mythril** [3] performs symbolic execution to explore attackable states. **Securify** [4] employs dependency patterns to verify contract correctness. Despite their power, these tools are heavy, require lengthy analysis times, and are not designed for instantaneous IDE feedback.

Dynamic analysis executes contracts in controlled environments to observe behavior. Tools like Manticore [5] can traverse complex execution paths but at considerable computational cost.

Popular IDE extensions, such as Juan Blanco’s Solidity extension [6], offer features like code completion, formatting, and compilation, yet provide limited real-time security diagnostics.

Solidify aims to bridge the gap between deep static analyzers and lightweight IDE tooling. The goal is not to replace Slither in CI/CD pipelines, but to supply a first line of defense that continuously shifts security detection to the left in the development lifecycle.

Table 1: Comparison of Existing Smart Contract Analysis Tools vs. Solidify

Tool	Analysis Method	Speed	Environment	Real-time Feedback
Slither	Static (SlithIR)	Slow	CI/CD	No
Mythril	Symbolic Execution	Slow	CI/CD	No
Securify	Dependency Patterns	Slow	CI/CD	No
Manticore	Dynamic Analysis	Very Slow	Testing Env.	No
Standard IDE Ext.	Syntax Checking	Fast	IDE	Limited
Solidify	Hybrid (AST + Regex)	Instant	IDE	Yes

METHODOLOGY

Meeting real-time performance targets while maintaining accuracy requires a hybrid and modular design.

Hybrid Analysis Architecture

Solidify blends two complementary analysis techniques:

1. **AST-based analysis:** We rely on the `tree-sitter` library [7] and the `tree-sitter-solidity` grammar to parse source code into an AST. `tree-sitter` is fast, incremental, and resilient to incomplete or syntactically invalid code-critical properties for real-time tooling. ASTs reveal structural and semantic information such as:
 - Determining variable scope to flag declarations that are never used.
 - Differentiating variable declarations from references.
 - Locating function bodies and control structures for rule evaluation.
2. **Regex-based analysis:** Traversing the AST for every lightweight rule can be overkill. For localized patterns, regular expressions are faster. We use regex for tasks like:
 - Detecting dangerous or deprecated keywords (`tx.origin`, `suicide`).
 - Enforcing naming conventions.
 - Spotting simple syntax mistakes, such as missing semicolons.

This hybrid approach leverages regex speed for simple checks and AST precision for context-aware rules.

System Architecture

Solidify follows a five-layer architecture in which each layer builds upon services provided by the one beneath it, isolating responsibilities and easing long-term maintenance:

- **Layer 1 - VS Code Integration:** The outer layer anchors the tool to the IDE lifecycle. It registers `TextDocument` listeners, manages activation/disposal, and provides debounce mechanisms when users type rapidly. This is the sole entry point to VS Code APIs.
- **Layer 2 - Core Analysis Engine:** The coordinator receives requests from Layer 1, normalizes inputs, manages caches, and orchestrates the analysis pipeline. It defines shared interfaces such as `RuleContext` and `RuleFinding` for upper layers.
- **Layer 3 - AST/CFG Processing:** This middle layer constructs structural views of the contract. Solidify invokes `tree-sitter` to build ASTs and infers basic control relationships (function boundaries, conditional blocks, loops) to emulate a lightweight control-flow graph (CFG). The resulting context objects allow rules to consume structured data without traversing the tree directly.

- **Layer 4 - Enhanced Rules Engine:** The modules that define the rules are resided here, each dedicated to a domain. With consistent interfaces from Layer 2 and context from Layer 3, rule authors focus on detection logic rather than infrastructure.
- **Layer 5 - Advanced Diagnostic Display:** The presentation layer converts findings into `DiagnosticCollection` entries, assigns severities, formats messages, and surfaces highlights, popovers, and the Problems panel. It is also the hook for future features like Quick Fixes or telemetry.

This layered architecture isolates change: adding rules affects Layer 4, optimizing AST/CFG logic touches Layer 3, while IDE integration remains stable in Layer 1.

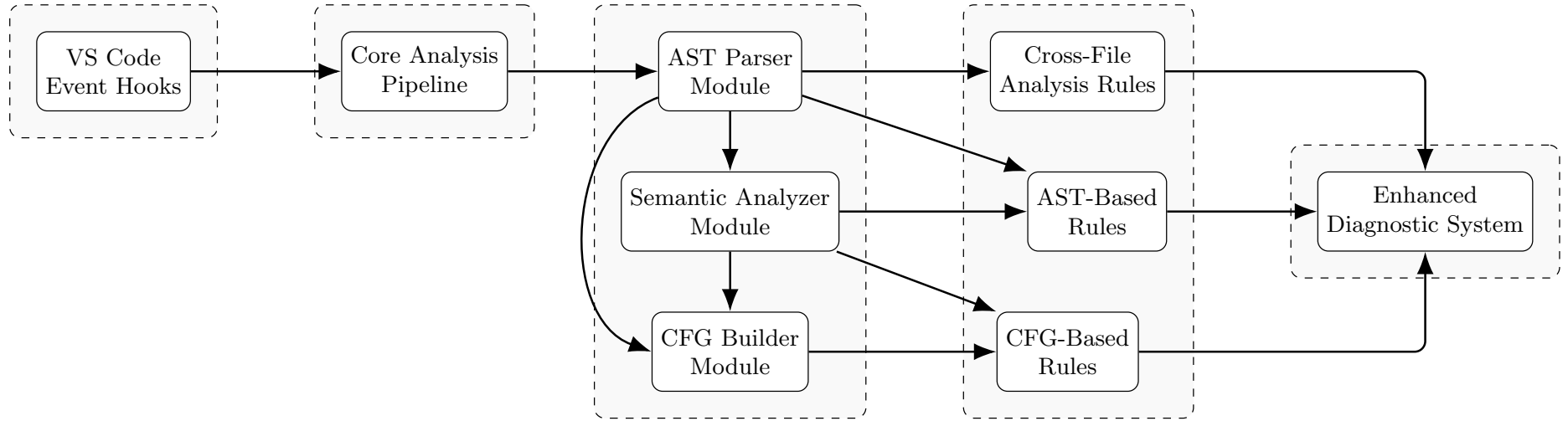


Figure 1: Solidify System Architecture

RESULTS

We assess Solidify along two axes: performance, measured by analysis latency, and accuracy, measured by standard security metrics (Precision, Recall, F1-Score).

Experimental Setup.

Dataset: We collect 20 real-world Solidity smart contracts of varying size and complexity. Contracts are sampled from popular open-source projects on GitHub to ensure diversity and relevance.

Baseline Tool: Slither serves as the baseline given its prominence and state-of-the-art status in Ethereum contract analysis.

Hardware Configuration: All experiments run on identical hardware (CPU: Intel i9-14900HX, RAM: 32 GB) to guarantee fair comparison.

Table 2: Comparison of Solidify vs. Slither Performance and Accuracy

Metric	Solidify	Slither
Analysis Latency	82ms (± 15 ms)	~ 2.5 s
Precision	0.61	0.91
Recall	0.59	0.94
F1 Score	0.60	0.92
False Positives	0.39	0.09

Response Latency.

The primary goal of Solidify is instantaneous feedback. We compare average analysis latency against Slither.

Solidify reaches 82 ms on average, well below the sub-100 ms human-interaction threshold. This is roughly $30\times$ faster than Slither (~ 2.5 s), illustrating our core contribution: transforming static analysis from a slow CI/CD batch step into an immediate IDE experience.

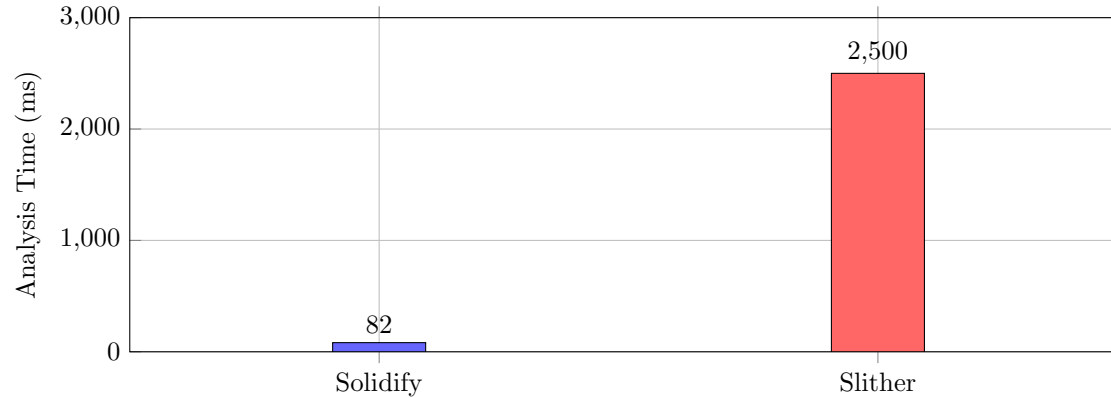


Figure 2: Average Analysis Latency Comparison

Accuracy Metrics.

We evaluate detection quality of Solidify versus Slither on the same dataset.

Slither secures a much higher F1 score (0.92 vs. 0.60). Its whole-program analysis yields rich control-flow graphs and sophisticated data-flow analyses that

suppress false positives. In contrast, Solidify favors hybrid AST/regex techniques geared for speed, accepting higher noise (Precision: 0.61, False Positives: 39%) in exchange for immediate responsiveness.

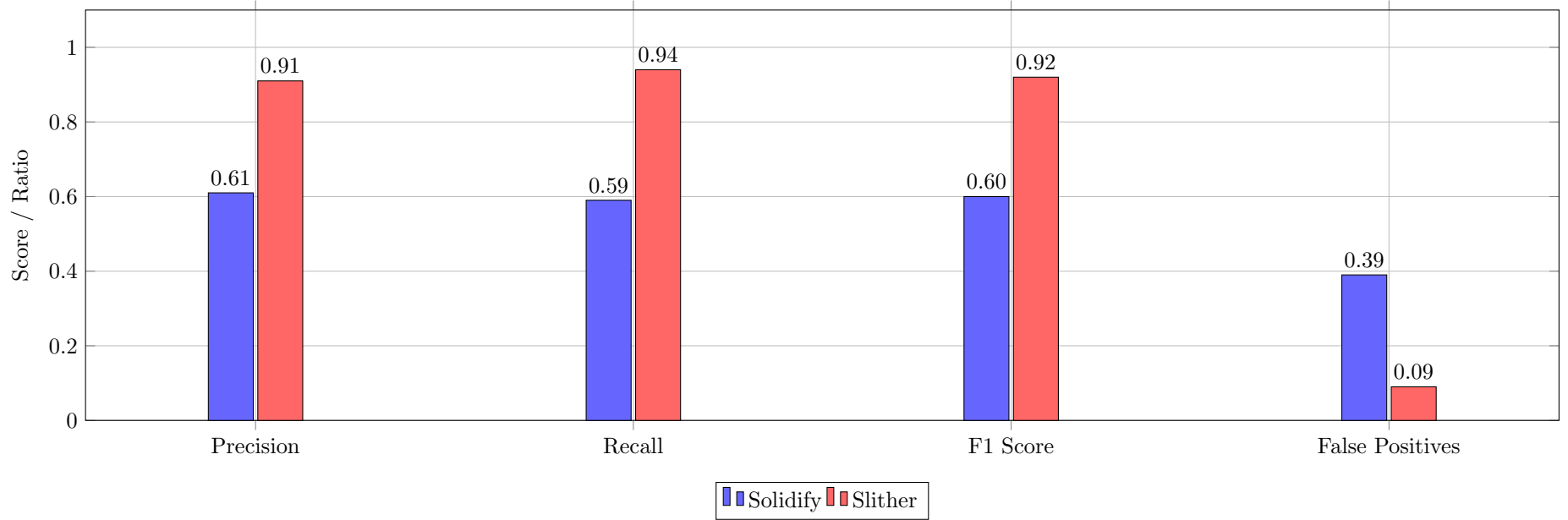


Figure 3: Comparative Analysis of Accuracy Metrics between Solidify and Slither

DISCUSSION

Solidify proves effective as a “first-line defense.” Deep IDE integration shifts security feedback from a deferred activity to a continuous part of editing.

Strengths.

The latency advantage stems from lightweight, scope-aware analysis and high-speed **tree-sitter** parsing, whereas Slither must build SlithIR and execute heavier analyses.

The lower accuracy ($F1 = 0.60$ vs. 0.92) is an intentional trade-off. Slither’s whole-program reasoning exposes complex vulnerabilities (e.g., multi-contract reentrancy). Solidify, in contrast, concentrates on syntax mistakes, local anti-patterns, and straightforward security hazards where instant feedback matters most. Our goal is to reduce developer-facing false positives early, not to supplant deep audits.

Limitations.

Solidify is not designed for cross-contract analysis or symbolic execution. It catches common, localized issues but cannot replace comprehensive project-wide analyses or formal audits. We recommend using Solidify alongside heavyweight tooling in CI/CD and traditional security reviews.

CONCLUSION

We presented Solidify, a real-time static analysis extension for Solidity inside VS Code. By combining AST reasoning and regex heuristics, we markedly shorten the feedback loop for developers. Experiments show 82 ms latency-30 $\times$ faster than CI/CD-oriented analyzers-while maintaining $F1 = 0.60$. This demonstrates the viability of trading analytical depth for speed to enhance developer experience and promote secure coding habits from the outset.

Future work includes:

- **Advanced rules:** Detect more sophisticated patterns such as reentrancy, integer overflow/underflow (with compiler assistance), and unsafe delegation flows.
- **Quick Fix integration:** Offer automated fixes, e.g., adding `payable` or removing unused variables.
- **Configurability:** Allow developers to tune severities or suppress specific diagnostics inline.
- **Performance research:** Further optimize analysis algorithms and introduce caching for very large projects.
- **Deeper analysis:** Incorporate selected intra-function data-flow analyses to curb false positives.

Empowering developers with intelligent, integrated, and instantaneous tooling is crucial to building a safer, more trustworthy Web3 ecosystem.

REFERENCES

- [1] N. Atzei, M. Bartoletti, and T. Cimoli, “A survey of attacks on Ethereum smart contracts,” in *Principles of Security and Trust (POST)*, ser. Lecture Notes in Computer Science, vol. 10260. Cham: Springer, 2017, pp. 109–139.
- [2] J. Feist, G. Grieco, and G. Mvel, “Slither: A static analysis framework for smart contracts,” in *IEEE International Conference on Blockchain (Blockchain)*, 2019, pp. 1–9.
- [3] B. Mueller, “Mythril: A security analysis tool for Ethereum smart contracts,” GitHub repository. [Online]. Available: <https://github.com/ConsenSysDiligence/mythril>
- [4] P. Tsankov et al., “Securify: Practical security analysis of smart contracts,” in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018, pp. 673–688.
- [5] M. Mossberg et al., “Manticore: A symbolic execution tool for analysis of smart contracts and binaries,” GitHub repository. [Online]. Available: <https://github.com/trailofbits/manticore>

- [6] J. Blanco, “Solidity extension for Visual Studio Code,” VS Code Marketplace, 2016. [Online]. Available: <https://github.com/juanfranblanco/vscode-solidity>
- [7] M. Brunsfeld, “Tree-sitter: A new parsing system for programming tools,” GitHub repository, 2018. [Online]. Available: <https://github.com/tree-sitter/tree-sitter>